Contents

1 vfl-renderer - Complete Documentation **1**
1.1 Repository Structure . . . 1
1.2 Architecture Overview . . . 2
1.2.1 Application Type & Runtime . . . 2
1.2.2 Two Independent Portals . . . 3
1.2.3 Entry Point & Middleware Pipeline . . . 3
1.2.4 Architecture Diagram . . . 3
1.3 Setup & Deployment . . . 4
1.3.1 Prerequisites . . . 4
1.3.2 Configuration — appsettings.json . . . 4
1.3.3 ApiSettings Quick Reference . . . 5
1.3.4 Data Protection . . . 5
1.3.5 Database . . . 6
1.3.6 Sitefinity CMS Integration . . . 6
1.3.7 CI/CD (Azure DevOps) . . . 6
1.3.8 IIS Deployment Steps . . . 6
1.4 Component Documentation . . . 6
1.4.1 Controllers . . . 6
1.4.2 Services . . . 12
1.4.3 HTTP Clients . . . 12
1.4.4 Entities (Sitefinity Widget Designer Properties) . . . 13
1.4.5 ViewComponents . . . 13
1.4.6 VFL.Renderer.OSApi Class Library . . . 13
1.5 API Reference . . . 14
1.5.1 Authentication Overview . . . 14
1.5.2 MyVodafone Endpoints . . . 14
1.5.3 MyBill Endpoints . . . 16
1.6 Testing Guide . . . 19
1.6.1 Current State . . . 19
1.6.2 Proposed Testing Strategy . . . 19
1.7 Security & Quality . . . 20
1.7.1 Critical Issues . . . 20
1.7.2 High Priority Issues . . . 21
1.7.3 Medium Priority Issues . . . 21
1.7.4 Code Quality Issues . . . 21

1 vfl-renderer - Complete Documentation

Version: 1.0.0 | **Date:** 2026-04-18 | **Framework:** ASP.NET Core 9 / Sitefinity 15.3

1.1 Repository Structure

```
vfl-renderer/
├── .gitignore
```

```
├── BannedSymbols.txt          # Banned API rules (HttpClient direct instantiation)
├── EULA.md
├── README.md
├── azure-pipelines.yml        # CI - triggered on main (VSBuild + VSTest)
├── azure-pipelines-1.yml      # CI - manual trigger (.NET dotnet publish)
├── nuget.config               # NuGet sources (nuget.org + nuget.sitefinity.com)
├── docs/
│   └── combined/              # This file + generated PDF
├── scripts/
│   └── build-docs-pdf.sh      # Pandoc PDF generation script
│
├── VFL.Renderer.OSApi/        # Standalone class-library wrapper (not used by WebApp)
│   ├── VFL.Renderer.OSApi.csproj
│   ├── Clients/
│   │   ├── IApiClient.cs
│   │   ├── BaseApiClient.cs
│   │   ├── AuthApiClient.cs
│   │   ├── RegistrationApiClient.cs
│   │   └── ResetPasswordApiClient.cs
│   ├── Common/BaseResponse.cs
│   ├── Config/ApiSettings.cs
│   └── LoginForm|Registration|ResetPassword Models
│
└── WebApp/                    # Main ASP.NET Core 9 Sitefinity Renderer
    ├── VFL.Renderer.csproj    # net9.0, Progress.Sitefinity 15.3
    ├── Program.cs             # Entry point, DI, middleware pipeline
    ├── appsettings.json       # Config (API URLs, Serilog, auth)
    ├── ApiClients/            # HTTP client wrappers
    ├── Common/                # Helpers (CookieHelper, EncryptionHelper, ApiResponse)
    ├── Config/ApiSettings.cs  # Strongly-typed config POCO
    ├── Controllers/           # 23 API controllers
    ├── Dto/                   # Simple DTOs
    ├── Entities/              # Sitefinity widget entity definitions (24 entities)
    ├── Extensions/            # ServiceCollectionExtensions, SitefinityServiceClient
    ├── Models/                # Widget view-model interfaces + implementations
    ├── Services/              # 14 service folders
    ├── ViewComponents/        # 31 ASP.NET ViewComponents (Sitefinity widget renderers)
    └── Views/Shared/Components/  # 35+ Razor .cshtml widget views
```

1.2 Architecture Overview

1.2.1 Application Type & Runtime

Item	Value
Application type	ASP.NET Core 9 Sitefinity Renderer
Framework	net9.0

Item	Value
Sitefinity version	15.3.8522.83
HTTP resilience	Polly (3-retry exponential + circuit-breaker)
Logging	Serilog → MS SQL Server (Logs table, auto-created)
Auth	Cookie-based, two independent schemes
Session	ASP.NET Core session (.MyBill.Session, 7-day idle timeout)

1.2.2 Two Independent Portals

The application hosts **two separate customer portals** on the same binary:

Portal	Auth Scheme	Backend Base URL	Purpose
MyVodafone	Cookies (default)	ApiSettings.BaseUrl	Online self-service (prepay/postpay)
MyBill	MyBillAuth	ApiSettings.MyBillBaseUrl	Postpaid billing portal

1.2.3 Entry Point & Middleware Pipeline

`WebApp/Program.cs` configures the pipeline in order:

1. Serilog -> MSSqlServer
2. `AddDataProtection` — persists keys to `./keys/` for consistent cookie encryption
3. `AddAuthentication` — two cookie schemes + JwtBearer (reads SitefinityJwtAuth cookie)
4. `AddAppServices()` — all DI registrations (ServiceCollectionExtensions.cs)
5. `AddSitefinity()` + `AddViewComponentModels()` + `AddFormViewComponentModels()`
6. **Middleware order:** UseStaticFiles -> UseRouting -> UseSession -> UseAuthentication -> UseAuthorization -> MyBill auth-selection inline middleware -> UseSitefinity()

The custom inline middleware (Program.cs lines 255-318) inspects `MyBillAuthCookie` on every request, re-authenticates via `MyBillAuth` scheme, and sets `context.User`, enabling Sitefinity-managed pages (not just API routes) to carry MyBill identity.

1.2.4 Architecture Diagram

```
Browser / SPA
      |
      | HTTPS
      v
+-----------------------------------------+
|      Middleware Pipeline                |
|  (Auth, Session, Sitefinity)            |
|                                         |
|  +-------------+   +----------------+   |
|  | Controllers |   | ViewComponents |   |
|  | (API layer) |   | (Widget layer) |   |
```

```
|  +------+------+   +-------+--------+   |
|         |                  |            |
|         +--------+---------+            |
|                  |                      |
|           +-----------+                 |
|           |  Services |                 |
|           +-----------+                 |
|                  |                      |
|         +--------+--------+             |
|         |   HTTP Clients  |             |
|         +--------+--------+             |
+------------------+---------------------+
                   |
        +----------+----------+
        |          |          |
        v          v          v
  VF Online    MyBill    Sitefinity
  Services     API       CMS
  API          GraphQL
```

Key external dependencies:

- **Sitefinity CMS** at `https://testfprn.vodafone.com.fj:4343` — page delivery, widget configuration
- **VF Online Services API** at `https://testonlineservices.vodafone.com.fj:4343` — MyVodafone backend
- **MyBill API** at `https://testmybill.vodafone.com.fj:4343` — MyBill backend (REST + GraphQL)
- **MS SQL Server** — Serilog structured log sink

1.3 Setup & Deployment

1.3.1 Prerequisites

Tool	Version
.NET SDK	9.0
Node.js (for wwwroot assets)	14.x or later
MS SQL Server	Any recent version
Sitefinity CMS	v15.3 (reachable on network)

1.3.2 Configuration — appsettings.json

```
{
  "Sitefinity": {
    "Url": "https://localhost:44352/",
    "WebServicePath": "api/default",
    "PropagateWidgetExceptions": true
```

```
  },
  "ApiSettings": {
    "BaseUrl": "https://testonlineservices.vodafone.com.fj:4343/",
    "MyBillBaseUrl": "https://testmybill.vodafone.com.fj:4343/",
    "MyBillLoginPath": "/my-bill-login",
    "MyBillLogoutPath": "/mybill-logout",
    "MyBillDashboardPath": "/mybill/mybill-dashboard",
    "MyBillProfilePath": "/mybill-profile",
    "MyBillTransactionHistoryPath": "/mybill-transactions",
    "MyBillResetPasswordPath": "/mybill-reset-password",
    "MyBillSupportLoginRedirectUrl": "/mybill/mybill-dashboard",
    "MyBillJwtExpiryMinutes": 30,
    "MyBillRefreshTokenExpiryMinutes": 10080,
    "RefreshTokenExpiryMinutes": 10080,
    "JwtExpiryMinutes": 30,
    "LoginPath": "/login"
  },
  "Serilog": {
    "WriteTo": [{
      "Name": "MSSqlServer",
      "Args": {
        "connectionString": "<your-connection-string>",
        "tableName": "Logs",
        "autoCreateSqlTable": true
      }
    }]
  }
}
```

1.3.3 ApiSettings Quick Reference

Property	Default	Description
BaseUrl	—	MyVodafone backend
MyBillBaseUrl	—	MyBill backend
MyBillJwtExpiryMinutes	14	JWT access token TTL
MyBillRefreshTokenExpiryMinutes	10080	Refresh token / cookie TTL (7 days)
RefreshTokenExpiryMinutes	10080	MyVodafone cookie TTL
LoginPath	/login	MyVodafone login page
MyBillLoginPath	/my-bill-login	MyBill login page
MyBillDashboardPath	/mybill/mybill-dashboard	Post-login redirect

1.3.4 Data Protection

Keys are persisted in `./keys/` relative to content root. For production ensure the IIS App Pool identity has write access to this directory, or configure a shared DPAPI store (Azure Key Vault / Redis) for multi-instance deployments.

1.3.5 Database

The application uses **no ORM or application-managed schema**. The only database interaction is the Serilog `MSSqlServer` sink, which auto-creates a `Logs` table on first run.

1.3.6 Sitefinity CMS Integration

The renderer acts as a headless front-end for Sitefinity:

- `AddSitefinity()` registers the Sitefinity SDK
- `UseSitefinity()` handles CMS-managed page routing and serves widget ViewComponents
- `Sitefinity:Url` must point to the running CMS instance
- Each widget = **Entity** (designer properties) + **Model** (view logic) + **ViewComponent** (glue) + **Razor .cshtml** (output)
- Widgets are placed on CMS pages via the Sitefinity Page Editor

1.3.7 CI/CD (Azure DevOps)

azure-pipelines-1.yml (manual trigger — recommended for releases):

```yaml
- task: DotNetCoreCLI@2
  inputs:
    command: publish
    projects: 'WebApp/VFL.Renderer.csproj'
    arguments: '--configuration Release --runtime win-x64 --self-contained true
                --output $(Build.ArtifactStagingDirectory)'
```

azure-pipelines.yml (triggers on `main`):

```yaml
- task: VSBuild@1
  inputs:
    msbuildArgs: '/p:DeployOnBuild=true /p:WebPublishMethod=Package ...'
- task: VSTest@2
```

1.3.8 IIS Deployment Steps

1. `dotnet publish -c Release -r win-x64 --self-contained`
2. Install the ASP.NET Core Hosting Bundle on the IIS server
3. Create an IIS site pointing to the publish output folder
4. Set App Pool to **No Managed Code**
5. Set `ASPNETCORE_ENVIRONMENT` environment variable (Production / Staging)
6. Grant the App Pool identity write access to `./keys/`
7. Configure HTTPS binding and SSL certificate

1.4 Component Documentation

1.4.1 Controllers

1.4.1.1 AccountController Handles OAuth external login (Google). Currently a stub — Google auth is commented out in Program.cs but the controller structure is present.

Method	Route	Auth	Description
GET	/Account/Login	None	Challenges Google auth
GET	/Account/GoogleResponse	Cookie	OAuth callback handler
GET	/Account/Logout	Cookie	Signs out cookie scheme

1.4.1.2 LoginFormController Route: `api/LoginFormController/[action]`

Method	Route	Auth	Description
POST	Login	None	Standard username/password login. Calls backend POST /api/Login/Standard. On success creates cookie principal with access_token claim. Sets OnlineServicesRefreshToken cookie from backend Set-Cookie header.
GET	Logout	None	Clears cookies, session, signs out
POST	GoogleCallback	None	Accepts Google ID token, calls backend /api/Login/Google

Request body (Login):

```
{ "username": "user@example.com", "password": "secret" }
```

Response 200:

```
{
  "data": { "isLoggedIn": true, "jwtToken": "eyJ...", "username": "user@example.com" },
  "StatusCode": 200
}
```

1.4.1.3 MyBillLoginFormController Route: `api/MyBillLoginFormController/[action]`

Method	Route	Auth	Description
POST	Login	None	MyBill login (BAN + password). Signs in under MyBillAuth scheme.
GET	Logout	None	Clears MyBillAuthCookie, session, redirects to login page
GET	RequestNewAccessToken	MyBillAuth	Refreshes JWT using backend refresh-token cookie
GET	LoginUsingSingleAccessToken	None	Support token login (JSON response)
GET	SupportLogin	None	Support token login (full HTML redirect page)

Request body (Login):

```
{ "username": "BAN123", "password": "secret" }
```

Side effects: forwards `Set-Cookie` (refresh token) from MyBill backend to browser; signs in `MyBillAuth` cookie valid for 7 days.

1.4.1.4 DashboardController

Route: `api/DashboardController/[action]` Auth: Implicit (reads `SitefinityJwtAuth` cookie via `WebClient`)

Method	Route	Description
GET	GetProfileInformation	Returns profile + device list; reads/writes luData encrypted cookie
GET	GetLuData	Returns raw luData cookie value
POST	GetBalanceInformation	Body: phone number string. Returns BalanceResponse.
POST	AddDevice	Body: DeviceRequest (decrypts oCresponse)
POST	RemoveDevice	Body: phone number string
POST	SendOTPCode	Body: phone number. Returns encrypted OTP code.

1.4.1.5 MyBillDashboardController

Route: `api/MyBillDashboard/[action]` Auth: `[Authorize(AuthenticationSchemes = "MyBillAuth")]`

Key endpoints:

- GET `GetAccounts` — BAN accounts from GraphQL
- GET `GetInvoices?selectedBan={ban}` — invoices for selected BAN
- GET `GetInvoiceDetails/{banNumber}/{invoiceNumber}` — invoice detail
- GET `GetProfileInformation` — cached profile info
- POST `SelectAccount` — switches selected BAN

1.4.1.6 MyBillCheckoutController

Route: `api/mybill-checkout` Auth: `[Authorize(AuthenticationSche = "MyBillAuth")]`

Method	Route	Description
POST	create-session	Creates checkout session. Validates: email, BAN, invoice, amount > 0, partial >= $10. Returns sessionId.
POST	submit	Form post with sid + payment method. Calls MyBillPaymentService. Returns redirect URL.

Create-session request:

```
{
  "email": "user@example.com",
  "banNumber": "BAN123",
```

```
  "invoiceNumber": "INV-001",
  "fullAmount": 150.00,
  "paymentAmount": 50.00,
  "isPartialPayment": true,
  "pageUrl": "https://example.com/mybill-checkout"
}
```

1.4.1.7 MyBillTransactionHistoryController

Route: `api/MyBillTransactionHistoryController/[acti`
Auth: `[Authorize(AuthenticationSchemes = "MyBillAuth")]` + explicit claim validation

Method	Route	Description
GET	GetBanAccounts	Returns BAN list from GraphQL
POST	GetTransactionHistory	Cursor-based pagination. Request body includes pageSize, after cursor, banNumber, status, invoiceNumber, dateFrom, dateTo.

GetTransactionHistory request:

```
{
  "pageSize": 20,
  "after": null,
  "banNumber": "BAN123",
  "status": "Success",
  "invoiceNumber": "INV-001",
  "dateFrom": "2024-01-01",
  "dateTo": "2024-12-31"
}
```

Response includes: `{ data: { transactions[], totalCount, pageInfo: { endCursor, hasNextPage } } }`

1.4.1.8 MyBillProfileSettingsController

Route: `api/MyBillProfileSettings/[action]`
Auth: `[Authorize(AuthenticationSchemes = "MyBillAuth")]`

Method	Route	Description
GET	GetProfileInfo/{selectedBan}?refresh=true	Returns account/contact info from GraphQL
PUT	UpdateProfile/{selectedBan}	Updates account name, contact name/phone, email
PUT	UpdatePassword/{selectedBan}	Updates password (includes newPassword, confirmPassword)

1.4.1.9 MyBillResetPasswordController

Route: `api/MyBillResetPasswordController/[action]`
Auth: None (public endpoints)

Method	Route	Description
POST	SendResetPasswordEmail	Body: { ban }. Triggers forgot-password email.
POST	ResetUserPassword	Body: { securityToken, newPassword }. Parses token/user from query string.

1.4.1.10 RegistrationWidgetController

Route: `api/RegistrationWidgetController/[action]`
Auth: None

Method	Route	Description
POST	Register	Body: RegistrationRequest (email, firstName, lastName, password, phone, verificationPageUrl)
POST	ConfirmEmailAsync	Body: EmailVerify (token, userId)
POST	ResendConfirmationEmail	Body: ResendEmailVerify (email)

1.4.1.11 ResetPasswordWidgetController

Route: `api/ResetPasswordWidgetController/[action]`
Auth: None

Method	Route	Description
POST	SendResetPasswordEmail	Body: ForgotPasswordRequest (email, verificationPageUrl)
POST	ResetUserPassword	Body: { securityToken, password }. Parses token/user from query string.

1.4.1.12 SubscriptionController

Route: `Subscription/[action]`

Method	Route	Description
POST	SendOtpRequest	Body: { number }. Returns encrypted OTP.
POST	Subscribe	Body: SubscriptionRequest (number, planId, oCresponse encrypted)
POST	Resubscribe	Same as Subscribe
POST	Unsubscribe	Same as Subscribe
GET	GetPlanById	Query: number, planId
GET	GetPlansByType	Query: number, planType
GET	GetAllPlans	Query: number

1.4.1.13 PurchasePlanController

Route: `PurchasePlan/[action]`

Method	Route	Description
GET	GetPlanById	Query: number, planId

Method	Route	Description
GET	GetPlansByType	Query: number, planType
GET	GetPlansByPaymentMethod	Query: number, paymentMethod
GET	GetPlansByTypeAndPaymentMethod	Query: number, planType, paymentMethod
GET	GetAllPlans	Query: number

1.4.1.14 TransactionHistoryController

Route: `api/TransactionHistoryController/[action]`

All endpoints accept a `GraphQLRequest` body with date/filter parameters:

- POST `GetWebTopUpHistory`
- POST `GetPurchasePlanHistory`
- POST `GetDirectTopUpHistory`
- POST `GetPlanActivationHistory`
- POST `GetPrepayGiftHistory`

1.4.1.15 ValidationController

Route: `api/ValidationController/[action]`

Method	Route	Description
POST	CheckEmailIsRegistered	Body: email string
POST	CheckNumberRegistered	Body: phone number string
POST	CheckNumberIsValid	Body: phone number string
POST	CheckNumberIsValid_AllowInactiveNumber	Body: phone number string
POST	CheckNumber	Combined validity + registration check

1.4.1.16 DirectTopUpController

Route: `api/DirectTopUpController/[action]`

Method	Route	Description
POST	SendRequest	Body: DirectTopUpRequest. No authentication required.

1.4.1.17 CommonController

Route: `api/CommonController/[action]`

Method	Route	Description
GET	GetCookie	Returns luData cookie value

1.4.1.18 CheckoutSessionController

Manages checkout session lifecycle for MyVodafone WebTopUp payments.

Note: `WebTopUpController` and `WebTopUpPublicController` are entirely commented out — their functionality is handled through `CheckoutSessionController`.

1.4.2 Services

Service	Interface	Key Operations
AuthService	IAuthService	LoginAsync, RefreshTokenAsync, ExternalLoginAsync, SingleAccessLoginAsync
AuthServiceMyBill	IAuthServiceMyBill	MyBillLoginAsync, RequestNewAccessTokenAsync, LoginUsingSingleAccessTokenAsync
DashboardService	IDashboardService	AddDevice, GetBalanceInformation, RemoveDevice, SendOTPCode
ProfileService	IProfileService	GetProfileInformationAsync
MyBillProfileService	IMyBillProfileService	GraphQL: GetAllAccountsByPrimary, GetInvoices; in-memory cache per user
RegistrationService	IRegistrationService	RegisterAsync, ConfirmEmailAsync, ResendConfirmationEmailAsync
ResetPasswordService	IResetPasswordService	SubmitRequest, ResetPassword
MyBillResetPasswordService	IMyBillResetPasswordService	SubmitRequest, ResetPassword via MyBill API
ValidationService	IValidationService	Phone/email validation via backend
SubscriptionService	ISubscriptionService	Plan listing, subscribe/resubscribe/unsubscribe, OTP
PlansService	IPlansService	GraphQL plan queries for purchase plan widget
TransactionHistoryService	ITransactionHistoryService	GraphQL for 5 transaction types
MyBillTransactionHistoryService	IMyBillTransactionHistoryService	Cursor-based GraphQL pagination
DirectTopUpService	IDirectTopUpService	POST to direct topup backend
PrepayGiftingService	IPrepayGiftingService	Prepay gifting flow
WebTopUpService	IWebTopUpService	ProcessPayment (auth + public), ProvideUpdate (auth + public)
MyBillPaymentService	IMyBillPaymentService	ProcessPaymentAsync to MyBill payment API

1.4.3 HTTP Clients

Client	Auth	Purpose
AuthApiClient	None	Login and token operations for both portals
RegistrationApiClient	None	User registration
WebClient	SitefinityJwtAuth cookie	Authenticated MyVodafone API calls
MyBillWebClient	access_tokenMyBill claim	Authenticated MyBill API calls (UseCookies=false to preserve Set-Cookie forwarding)
MyBillPublicApiClient	None	Unauthenticated MyBill ops (forgot password)

Client	Auth	Purpose
ApiClient	Polly-wrapped	Generic client with retry + circuit-breaker

1.4.4 Entities (Sitefinity Widget Designer Properties)

Entities define what a content editor can configure in the Sitefinity Page Builder:

Entity	Configurable Properties
CustomLoginFormEntity	Login/logout page URLs, error messages
DashboardEntity	API endpoint overrides, redirect URLs
MyBillLoginFormEntity	Login paths, MyBill API settings
CheckoutEntity	Payment URLs, redirect settings
WebTopUpEntity	Payment provider settings
HeroBannerSlideEntity	Banner content, image references
PricePackageEntity	Package display configuration
SubscriptionEntity	Plan type filters

1.4.5 ViewComponents

Each ViewComponent is a Sitefinity widget renderer:

1. Decorated with `[SitefinityWidget]` from Progress.Sitefinity.AspNetCore
2. Receives the Entity (designer properties) in InvokeAsync
3. Calls the Model to get data
4. Returns a view pointing to `Views/Shared/Components/{Name}/Default.cshtml`

ViewComponents with guarded access (show Unauthorized.cshtml when unauthenticated):

- MyBillDashboardViewComponent
- MyBillProfileSettingsViewComponent
- MyBillTransactionHistoryViewComponent
- CustomLoginFormViewComponent (redirects authenticated users away)

Full ViewComponent list (31 total):

CTA, Captcha3, Checkout, Child, CustomLoginForm, CustomMyBillLoginForm, CustomMyBillResetPassword, CustomProfile, CustomRegistration, CustomResetPassword, Dashboard, DirectTopUp, HeroBannerSlide, LoginStatus, MyBillCheckout, MyBillDashboard, MyBillLoginStatus, MyBillProfileSettings, MyBillSupportLogin, MyBillTransactionHistory, PageTitle, PrepayGifting, PricePackage, PurchasePlan, PurchasePlanPublic, StaticSection, Subscription, TransactionHistory, VitiProduct, WebTopUp, WebTopUpPublic

---

1.4.6 VFL.Renderer.OSApi Class Library

A standalone class library (not currently referenced by WebApp) that mirrors the WebApp API client pattern. It contains its own copies of:

- `BaseApiClient` / `IApiClient`
- `AuthApiClient` (login + token refresh)
- `RegistrationApiClient` (create, confirm email, check registered)
- `ResetPasswordApiClient` (submit request, reset password)
- `ApiSettings`, `BaseResponse`, LoginForm / Registration / ResetPassword models

Known bugs in this library:

- `ResetPasswordApiClient` constructor is incorrectly named `PasswordResetApiClient` (compile-time error if instantiated)
- `BaseApiClient.PutAsync` calls `PostAsJsonAsync` instead of `PutAsJsonAsync`

1.5 API Reference

1.5.1 Authentication Overview

Portal	Scheme	Cookie Name	Claim	Lifetime
MyVodafone	Cookies	SitefinityJwtAuth	access_token	30 min sliding
MyBill	MyBillAuth	MyBillAuthCookie	access_tokenMyBill	7 days fixed

JWT Refresh (MyBill): A background JS script (`mybill-auth-refresh.js`) calls `RequestNewAccessToken` before the 14-minute JWT expires. The backend uses the `myBillRefreshToken` cookie (forwarded at login) to issue a new JWT. The MyBillAuthCookie (7 days) outlives many JWT cycles.

1.5.2 MyVodafone Endpoints

1.5.2.1 POST /api/LoginFormController/Login

Authenticates a MyVodafone user.

Request:

```
{ "username": "user@example.com", "password": "secret" }
```

Response 200:

```
{
  "data": { "isLoggedIn": true, "jwtToken": "eyJ...", "username": "user@example.com" },
  "StatusCode": 200
}
```

Side effects: Sets `OnlineServicesRefreshToken` (HttpOnly, Secure, Lax, 7 days) from backend `Set-Cookie`.

1.5.2.2 GET /api/LoginFormController/Logout

Clears cookies, session, signs out. Redirects to `/`.

1.5.2.3 POST /api/LoginFormController/GoogleCallback

Request:

```
{ "idToken": "google-id-token" }
```

1.5.2.4 GET /api/DashboardController/GetProfileInformation?selectedNumber={number}

Returns profile + device list. Writes `luData` encrypted cookie.

Response 200:

```
{
  "data": {
    "firstName": "John", "lastName": "Doe", "email": "john@example.com",
    "devices": [{ "number": "7123456", "isSelected": true }]
  }
}
```

1.5.2.5 POST /api/DashboardController/GetBalanceInformation

Body: `"7123456"` (phone number as plain string)

1.5.2.6 POST /api/DashboardController/AddDevice

Body:

```
{ "number": "7123456", "oCresponse": "<encrypted-otp>" }
```

1.5.2.7 POST /api/DashboardController/SendOTPCode

Body: `"7123456"`

Response includes encrypted OTP code in `data.code`.

1.5.2.8 POST /api/RegistrationWidgetController/Register

Request:

```
{
  "email": "user@example.com", "firstName": "John", "lastName": "Doe",
  "password": "P@ss1", "phone": "7123456", "verificationPageUrl": "/verify"
}
```

1.5.2.9 POST /api/RegistrationWidgetController/ConfirmEmailAsync

Body:

```
{ "token": "...", "userId": "..." }
```

1.5.2.10 POST /api/ResetPasswordWidgetController/SendResetPasswordEmail

Body:

```
{ "email": "user@example.com", "verificationPageUrl": "/reset-confirm" }
```

1.5.2.11 POST /api/ResetPasswordWidgetController/ResetUserPassword

Body:

```
{ "securityToken": "?token=abc&user=xyz", "password": "NewP@ss1" }
```

1.5.2.12 POST /api/ValidationController/CheckEmailIsRegistered

Body: `"user@example.com"`

1.5.2.13 POST /api/ValidationController/CheckNumber

Combines validity + registration check. Body: `"7123456"`

1.5.2.14 POST /Subscription/Subscribe

Body:

```
{ "number": "7123456", "planId": 42, "oCresponse": "<encrypted-otp>" }
```

1.5.2.15 GET /PurchasePlan/GetPlansByType?number=7123456&planType=prepay

Returns array of plans.

1.5.2.16 POST /api/TransactionHistoryController/GetWebTopUpHistory

Body: GraphQLRequest with date range and filter fields.

1.5.3 MyBill Endpoints

1.5.3.1 POST /api/MyBillLoginFormController/Login

Request:

```
{ "username": "BAN123", "password": "secret" }
```

Response 200:

```
{
  "data": { "isLoggedIn": true, "jwtToken": "eyJ..." },
  "StatusCode": 200
}
```

Side effects: Forwards `Set-Cookie` (refresh token) from MyBill backend; signs in `MyBillAuth` cookie (7 days).

1.5.3.2 GET /api/MyBillLoginFormController/RequestNewAccessToken

Auth: MyBillAuth required. Refreshes JWT using backend refresh-token cookie.

1.5.3.3 GET /api/MyBillLoginFormController/SupportLogin?token={token}

No auth required. Returns HTML page with error or redirects authenticated user.

1.5.3.4 POST /api/mybill-checkout/create-session

Auth: MyBillAuth

Request:

```
{
  "email": "user@example.com",
  "banNumber": "BAN123",
  "invoiceNumber": "INV-001",
  "fullAmount": 150.00,
  "paymentAmount": 50.00,
  "isPartialPayment": true,
  "pageUrl": "https://example.com/mybill-checkout"
}
```

Validation rules: - email required - banNumber required - invoiceNumber required - paymentAmount > 0 - paymentAmount <= fullAmount - if isPartialPayment: paymentAmount >= 10

Response:

```
{ "sessionId": "3fa85f64-5717-4562-b3fc-2c963f66afa6" }
```

1.5.3.5 POST /api/mybill-checkout/submit

Auth: MyBillAuth Content-Type: multipart/form-data

Fields: `sid`, `PaymentMethod` (MPAISA | CARD), billing details, `PaymentAmount?`, `IsPartialPayment?`

1.5.3.6 POST /api/MyBillTransactionHistoryController/GetTransactionHistory

Auth: MyBillAuth

Request:

```
{
  "pageSize": 20,
  "after": null,
  "banNumber": "BAN123",
  "status": "Success",
  "invoiceNumber": "INV-001",
  "dateFrom": "2024-01-01",
```

```
  "dateTo": "2024-12-31"
}
```

Response:

```
{
  "data": {
    "transactions": [
      {
        "transactionId": "TXN001",
        "invoiceNumber": "INV-001",
        "date": "2024-06-15",
        "amount": 50.00,
        "paymentMethod": "CARD",
        "status": "Success"
      }
    ],
    "totalCount": 142,
    "pageInfo": { "endCursor": "cursor==", "hasNextPage": true }
  },
  "isSuccess": true,
  "statusCode": 200
}
```

1.5.3.7 GET /api/MyBillProfileSettings/GetProfileInfo/{selectedBan} Auth: MyBillAuth

Response:

```
{
  "isSuccess": true,
  "data": {
    "accountName": "Acme Corp",
    "contactFullName": "John Doe",
    "phoneNumber": "7123456",
    "email": "john@example.com",
    "banNumber": "BAN123"
  }
}
```

1.5.3.8 PUT /api/MyBillProfileSettings/UpdateProfile/{selectedBan} Auth: MyBillAuth

Body:

```
{
  "accountName": "Acme Corp",
```

```
  "contactFullName": "John Doe",
  "contactPhoneNumber": "7123456",
  "email": "john@example.com"
}
```

1.5.3.9 PUT /api/MyBillProfileSettings/UpdatePassword/{selectedBan}

Auth: MyBillAuth

Body:

```
{ "newPassword": "NewP@ss1!", "confirmPassword": "NewP@ss1!" }
```

1.5.3.10 POST /api/MyBillResetPasswordController/SendResetPasswordEmail

Auth: None

Body:

```
{ "ban": "BAN123" }
```

1.5.3.11 POST /api/MyBillResetPasswordController/ResetUserPassword

Auth: None

Body:

```
{ "securityToken": "?token=abc&user=BAN123", "newPassword": "NewP@ss1!" }
```

1.6 Testing Guide

1.6.1 Current State

There are **no automated tests** in this repository. There are no `*Test*.csproj` files, no xUnit/NUnit/MSTest packages, and the `VSTest` step in `azure-pipelines.yml` will find no test assemblies to run.

1.6.2 Proposed Testing Strategy

1.6.2.1 Unit Tests (xUnit + Moq)

Priority areas:

1. **AuthApiClient** — mock HttpMessageHandler, verify cookie parsing in LoginAsync / MyBillLoginAsync, verify Set-Cookie forwarding logic
2. **MyBillLoginFormController** — mock IAuthServiceMyBill, test JWT claim extraction (ExtractUsernameFromJwt), edge cases with malformed tokens
3. **MyBillCheckoutController.CreateSession** — all validation rules (amount > 0, partial >= $10, paymentAmount <= fullAmount)
4. **CheckoutSessionStore** — Save / Get / Remove / expiry (singleton)
5. **EncryptionHelper / CookieHelper** — AES round-trip, cookie option verification

6. **ValidationController** — combined CheckNumber flow

Add to a new `WebApp.Tests` project:

```
<PackageReference Include="xunit" Version="2.9.0" />
<PackageReference Include="Moq" Version="4.20.72" />
<PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="9.0.0" />
```

1.6.2.2 Integration Tests (WebApplicationFactory)

Use `Microsoft.AspNetCore.Mvc.Testing` to spin up the full pipeline with mocked HTTP backends. Test the complete login -> cookie issued -> protected endpoint accessible flow.

1.6.2.3 Coverage Gaps

Gap	Risk
MyBill auth middleware (Program.cs lines 255-318)	High — stale cookie deletion could regress
EncryptionHelper.Decrypt with tampered input	High — OTP decryption failures silently propagate
Partial payment validation	Medium
ExtractUsernameFromJwt with malformed tokens	Medium
Polly retry/circuit-breaker behavior	Low-medium
GraphQL cursor pagination correctness	Low

1.7 Security & Quality

1.7.1 Critical Issues

1. Database credentials in source code (appsettings.json)

The connection string contains a hardcoded `sa` password. Rotate immediately and move to an environment variable, Azure Key Vault, or .NET Secret Manager.

```
"connectionString": "data source=localhost;UID=sa;PASSWORD=...;initial catalog=..."
```

2. SSL certificate validation disabled in Development (Program.cs lines 76-79, 210-213)

```
handler.ServerCertificateCustomValidationCallback = (msg, cert, chain, errs) => true;
```

Acceptable only in local dev. Ensure `IsDevelopment()` guard is never bypassed in staging or production.

3. Hardcoded backend URLs in ServiceCollectionExtensions.cs (lines 69, 78)

```
client.BaseAddress = new Uri("https://testonlineservices.vodafone.com.fj:4343/");
```

These override `ApiSettings.BaseUrl` loaded from configuration. Should read from `IOptions<ApiSettings>` instead.

1.7.2 High Priority Issues

4. No CSRF protection on POST endpoints

Most form-posting controllers have `[ValidateAntiForgeryToken]` commented out (RegistrationWidgetController, ResetPasswordWidgetController, LoginFormController). This exposes POST endpoints to CSRF from same-site pages.

5. CommonController.GetCookie and DashboardController.GetLuData

Both return the `luData` cookie value without any authentication requirement. This exposes profile data (name, email, device list) to unauthenticated callers.

6. Refresh token stored in shared IMemoryCache

`AuthService` uses a single `RefreshTokenKey = "Auth.RefreshToken"` in the process-wide memory cache, meaning refresh tokens from different users can overwrite each other in a multi-user scenario. Use per-user cache keys scoped by session or user ID.

7. AES encryption key — likely hardcoded

`EncryptionHelper` uses AES for OTP values and cookie data. If the key/IV is a hardcoded constant, all encrypted values can be decrypted by anyone with source access. Move the key to configuration / Key Vault.

1.7.3 Medium Priority Issues

8. No rate limiting on authentication endpoints

`/api/LoginFormController/Login` and `/api/MyBillLoginFormController/Login` have no throttling, making them vulnerable to credential-stuffing attacks. Add ASP.NET Core rate limiting middleware (`AddRateLimiter`) or a WAF rule.

9. No CORS policy

`AllowedHosts: "*"` in appsettings is for Kestrel host filtering, not CORS. Without explicit `AddCors` + `UseCors`, cross-origin AJAX from Sitefinity-hosted JavaScript will fail or, if Sitefinity sets permissive CORS, the API may be unintentionally exposed. Define an explicit CORS policy.

10. Potential XSS in SupportLogin HTML generator

`GenerateErrorHtml` uses `HtmlEncode` on `title` but passes `message` raw into `<p>`. If message content is attacker-influenced, this is reflected XSS. HtmlEncode all user-influenced strings.

11. SecurityToken parsing in ResetUserPassword

`request.SecurityToken.TrimStart('?').Split('&')` does not correctly handle base64 padding characters (`=`) that appear inside the token value. Use `HttpUtility.ParseQueryString` or `QueryHelpers.ParseQuery` instead.

1.7.4 Code Quality Issues

12. Dead code

- `WebTopUpController.cs` — entirely commented out (~250 lines)
- `WebTopUpPublicController.cs` — entirely commented out (~150 lines)

- `CheckoutController.cs` — entirely commented out

13. VFL.Renderer.OSApi not referenced

The `VFL.Renderer.OSApi` project is not referenced by `WebApp` and duplicates models and clients. Either integrate it as a shared package or remove it.

14. SubscriptionController missing logger assignment

The `_logger` field is declared but never assigned (missing constructor parameter), causing `NullReferenceException` in exception catch blocks.

15. Session/cookie lifetime mismatch

Session idle timeout is 7 days but the `SitefinityJwtAuth` cookie expires in 30 minutes (sliding). Session outlives the auth cookie by a wide margin, potentially leaving orphaned session state.

16. Singleton CheckoutSessionStore — not distributed

`ICheckoutSessionStore` is registered as Singleton with in-memory storage. In multi-instance deployments (load-balanced IIS), sessions created on one instance will not be found on another. Replace with a distributed cache (Redis) for production.

17. Service duplication

`DashboardService` is registered twice in `Program.cs` (lines 96-98 and 222-224) and `CustomLoginFormModel` is also registered twice. Remove duplicates.

End of Documentation — vfl-renderer v1.0.0 - Release Note 2

For full details, see the individual documents included here.